

V16 — אינטגרציה באפליקציית Kitors + סקירת קוד

מה נוסף כדי ש-V16 ירוץ על השעון, ומה נמצא בסקירה של 25,246 שורות ה-Watch App
1.8.2026 · מבוסס research/WatchOS-surfhub-main (objectVersion 77 / Xcode 16, Kitors.xcodeproj) · נסקרו: MotionManager, SessionManager, SessionLogger, JumpDetector*, JumpEngine*, ReplaySessionController, Tools/JumpReplay

תיקון לדיווח קודם שלי — חשוב לקרוא ראשון. בסקירה קודמת דיווחתי ש"השעון לא מריץ V15 אלא גלאי ישן (JumpDetect.swift)". **הדיווח הזה היה על ריפויטורי אחר** — surfhub-watch/watchos/SurfHubWatch — Watch App/, אפליקציה ישנה ופשוטה יותר בריפו שלכם.

הקוד האמיתי של צוות השעון (Kitors) תקין: V15 ממומש במלואו — JumpEngineV15.swift (1,761 שורות) + JumpDetectorV15.swift כאדפטר, מחווט ל- DetectionEngine.v15Clean, עם ניהול ברומטר single-consumer, יישור שעונים, ותורי-עבודה נפרדים. גם V12/V13/V14 מחווטים כך. אני מתנצל על הבלבול — ההמלצות שנתתי אז ("להוסיף ל-target", "להחליף batch ב-streaming") **אינן רלוונטיות כאן**; הן כבר קיימות.

1. מה נוסף עבור V16

קובץ / מקום	שינוי
Services/JumpEngineV16.swift	חדש — המנוע (585 שורות), תאום מלא של jumpEngineV16.ts
Services/JumpDetectorV16.swift	חדש — אדפטר JumpDetecting בסגנון V15
Models/Session.swift	displayName + description + case v16BigAir
Services/SessionManager.swift	makeJumpDetector + הפעלת ה-IMU המיידית
Services/ReplaySessionController.swift	makeDetector
Tools/JumpReplay/.../main.swift	"v16" / "v16-big-air"

הפרויקט משתמש ב- PBXFileSystemSynchronizedRootGroup (Xcode 16) — קבצים בתיקייה נכללים אוטומטית, אין צורך לערוך את ה-pbxproj.

⚠️ המלכות מספר 1 באינטגרציה — דומיין ה-g

popMinG = 1.4 של V16 מכויל בדומיין ה-מגניטודה: |userAcceleration|, שהוא ~0 g במנוחה — כלומר IMUSample.accelerationMagnitude.
הוא אינו JumpDetectorV15.verticalLoadG, שהוא העומס המוטל על ציר הכבידה ומראה ~1 g במנוחה:

```
// V15 (load domain, ~1 g at rest) – NOT what V16 wants
projected = (a . ĝ) / |ĝ| ; load = projected + |g|

// V16 (magnitude domain, ~0 g at rest)
loadG = sample.accelerationMagnitude
```

הזנת ה-load ל-V16 תזיז כל פופ בגרם שלם ותהפוך את סף ה-1.4 לחסר-משמעות. הכתובת מתועדת בראש JumpDetectorV16.swift.

1.1 מה V16 צורך ומה לא

שימוש ב-V16	ערוץ
טריגר הפופ	IMU: accelerationMagnitude
הערוץ האנכי — מדף העילוי והגובה	IMU: userAcceleration + צירי attitudeQuaternion
מטריקות בלבד (מהירות המראה, מרחק). אין שער-זיהוי שקורא אותו	GPS
אינו נצרך כלל — processAbsoluteAltitude נשאר ה-no-op של הפרוטוקול	ברומטר (abs / rel / לחץ)

IMUSample כבר נושא attitudeQuaternion ואת שלושת הצירים — לא נדרש שינוי ב-MotionManager.

2. ממצאי הסקירה

logSample הוא no-op בכל 7 האדפטרים

חומרה 1

```
// SessionLogger.swift:228
func logSample(sample:speed:baselinePressure:lowGCount:state:, event: String = "") {
    guard !event.isEmpty else { return } // יוצא מיד ←
    ...
}

// JumpDetectorV15.swift:244 (גם V7,V10,V11,V12,V13,V14)
let snapshotState = currentState() // NSLock
if snapshotState != .idle || sampleCount % 5 == 0 {
    SessionLogger.shared.logSample(sample:..., state: snapshotState.rawValue)
    // ברירת מחדל "" → כלום → event: אין ↑
}
```

נבדק: 7 מתוך 7 קריאות SessionLogger.shared.logSample(...) באדפטרים הן ללא event, כלומר כולן no-op. שורות ה-MOTION מגיעות ללוג דרך logMotionSamples מנתיב ה-batch של MotionManager — לא מכאן.

העלות: נעילת NSLock + מודולו + קריאת פונקציה בכל דגימה. ב-200 Hz זה 200 נעילות לשנייה של עבודה מבוצעת על שעון עם סוללה. חמור מכך — קורא הקוד חושב שהדגימות נרשמות, וזה מטעה בדיבאג. **תיקון:** להסיר את הבלוק מכל האדפטרים, או להוסיף event: אמיתי אם הכוונה הייתה לרשום. V16 כבר נמסר בלעדיו, עם הערה מסבירה.

שדה ה-precision נזרק — והוא שער-הבריאות היחיד שעובד

חומרה 1

```
// JumpDetectorV15.swift:279
func processAbsoluteAltitude(..., precisionM: Double?) {
    _ = precisionM // נזרק במפורש ←
    ...
    engine.addAbsoluteAltitude(t:..., altitudeM:..., accuracyM: sample...Accuracy)
}
```

המידדה שלנו על log_287 (7,612 דגימות abs):

מצב אמיתי	שמדוח accuracy	ערכים ייחודיים	דגימות	precision
קפוא	2.9 – 0.002 ("מצוין")	2 (0%)	968	5
חי ב-3 Hz	9.5 ("גרוע")	5,339 (80%)	6,644	0.5

accuracy הפוך למציאות. המנוע מסנן על $accuracyM \geq 12$ — כלומר הוא בוטח בדאטה הקפוא (שמדוחו 0.002) ויזרוק את הדאטה הטוב אם הסף יוקשח. $precision \leq 1$ הוא המבחן הנכון. זה מסביר מדוע Surfr, על אותה חומרה, מדד 11 מתוך 12 קפיצות שאצלנו הברומטר היה קפוא בהן לגמרי. **תיקון מוצע ל-V15:** להעביר את precisionM ל-addAbsoluteAltitude ולהתייחס ל- $precision > 1$ כערור (בדיוק כפי שהמנוע כבר מתייחס ל-frozen).

שני bootWallClock → עצמאיים → סחיפת שעונים בין GPS ל-IMU

חומרה 2

```

MotionManager.swift:337 let bootWallClock = Date().timeIntervalSince1970 - ProcessInfo.systemUptime
JumpDetectorV15.swift:80 let bootWallClock = Date().timeIntervalSince1970 - ProcessInfo.systemUptime //
נלכד שנית ←

```

הכמות `systemUptime` **אינה קבועה**: `systemUptime` אינו מתקדם בשינה עמוקה, והשעון הקירי מתוקן ע"י NTP. שתי לכידות ברגעים שונים נותנות ערכים שונים.

- **IMU** נכנס למנוע בזמן הגולמי `motion.timestamp` (דומיין ה-uptime) — נכון.
- **GPS** נכנס דרך `monotonicTime(from: CLLocation.timestamp)` — כלומר **uptime→wall** לפי ה-`bootWallClock` של האדפטר.

לכן כל סחיפה בין הרגע שבו נבנה ה-`MotionManager` לרגע שבו נבנה האדפטר, ועוד כל שינה שהצטברה מאז, מופיעה כהסטה בין ציר הזמן של ה-GPS לזה של ה-IMU.

השפעה: ב-V16 — GPS הוא מטריקות בלבד; הסטה מעל 3 שניות פשוט תפיל את ההתאמה `takeoffSpeedMS / distanceM` יחזרו nil (הידרדרות נקייה). **ב-V15 זה חמור יותר** — שם ה-GPS משתתף בשערים.

תיקון מוצע: מקור-אמת יחיד לשעון. או להעביר את ה-`CLLocation.timestamp` דרך אותו נרמול שכבר קיים ל-`abs (TimestampNormalizer)`, או להזריק את ה-`bootWallClock` של `MotionManager` לאדפטרים במקום שכל אחד ילכוד משלו. הצוות כבר בנה `alignedAbsoluteTimestamp` לבעיה זהה בברומטר — אותו פתרון צריך לחול על ה-GPS.

~6,150 שורות מנועים בלתי-נגישים בבינארי

חומרה 3

קובץ	שורות	נגיש מ- <code>DetectionEngine</code> ?
<code>KitesurfJumpEngineV11.swift</code>	2,241	כן (v11 — ברירת המחדל)
<code>KitesurfJumpEngine.swift</code>	1,324	כן (v7)
<code>KitesurfJumpEngineV10.swift</code>	1,088	כן (v10)
<code>KitesurfJumpEngine_old.swift</code>	998	לא
<code>KitesurfJumpEngineV8/V9.swift</code>	503	כן (v8/v9)

`_old` אינו נגיש מאף מסלול. מעבר לזה — האם V7–V11 עדיין נחוצים בייצור? כל אחד גורר קוד, זמן קומפילציה, ומשטח-בדיקה. **המלצה**: להעביר את הישגים ל-target של כלי-מחקר בלבד, ולהשאיר בבינארי של השעון את + V15 V16 (sensorRecorder +).

חומרה 4

נבדק ונמצא תקין — ללא ממצא

נושא	ממצא
sampleBuffer MotionManager	ב- בטוח — כל הגישות דרך motionProcessingQueue שהוא DispatchQueue טורי; stopTracking / pauseTracking משתמשים ב- sync. נכון
מחזורי-שמירה (retain cycles)	103 שימושי [weak self] — הסגורים הבורחים מטופלים עקבית
OperationQueue לסנסורים	maxConcurrentOperationCount = 1 על שלושתם — סריאליזציה נכונה
נפילה CMBatchedSensorManager	מטופלת: watchdog של 5s + didFallbackFromBatchedError מונע לולאה
guard altimeter	generation על ה- generation == self.absoluteStreamGeneration restart — מונע דלף מסטרים ישן אחרי

3. סדר פעולות מומלץ

- 1. `precision` ל-V15 — התיקון בעל התשואה הגבוהה ביותר. הוא לבדו עשוי להחזיר את הברומטר לחיים בסשנים שבהם היום הוא קפוא, וזה בדיוק הפער מול `Surfr`.
- 2. להסיר את `logSample` המת מ-7 האדפטרים — חיסכון מיידי של 200 נעילות/שנייה.
- 3. לאחד את שעון ה-GPS עם שעון ה-IMU (מקור `bootWallClock` יחיד או נרמול).
- 4. לבנות ולבדוק את V16 — ראה §4.
- 5. לנקות מנועים ישנים מהבינארי של השעון.

4. בדיקת קבלה ל-V16

אין לי Xcode על הסביבה הזו, ולכן V16 לא עבר קומפילציה — אימתי סטטית שכל סימבול שבשימוש קיים `JumpDetectorV13Readiness`, `Jump.init(sessionId:startTime:)`, `DetectionMode`, `MotionQuaternion`, `attitudeQuaternion`, `IMUSample.accelerationMagnitude` ושהחתימות תואמות. הצעד הראשון אצלכם: `build`.
לאחר מכן, ריצת ה-CLI על הלוגים הידועים — הערכים חייבים לצאת זהים לריפליי ה-TS:

```
swift run JumpReplay --engine v16 --log log_287_20260728_182822_05D47F19.kslog
```

לוג	קפיצות אמת	ציפייה מ-V16
log_287 (ביג-אייר 2.1m–8.5m)	14	14 · MAE גובה 0.52m · פנטום אחד
log_neg (פופים וגלים, אפס קפיצות)	0	0 פליטות
log_clean (קפיצות קטנות)	4	2 — מחוץ לתחום V15, V16 עדיף שם

אם המספרים לא תואמים — הסיבה הראשונה לבדוק היא דומיין ה-g (1\$). סימן היכר: אפס פליטות בכלל, או פליטות בכל פופ.

5. מה V16 לא פותר

מגבלה	סטטוס
airtime	MAE 0.54s מול 0.78s של מנבא-קבוע — כלומר בקושי מנצח קבוע. אין להשתמש בו כשער; מוצג עם הסתייגות
מרחק קפיצה	יורש את שגיאת ה-airtime: 6.27m. עם airtime נכון היה 2.78m
מהירות המראה	0.64 m/s — אמין, נמדד ישירות מה-GPS
קפיצות > 2.5m	מדף העילוי שלהן (0.2–0.6s) חופף לרעש. לנתב ל-V15
מעל 8.5m	לא מאומת — חלון קבוע של 4.5s לא יכסה קפיצה של 25 שניות

התכן הנכון הוא ניתוב, לא החלפה: V16 לביג-אייר, V15 לקפיצות קטנות. V15 משיג 11/12 בסשנים הקטנים בעזרת מפרידי GPS שאינם תלויים באורך המדף; V16 משיג 14/14 בביג-אייר בלי ברומטר בכלל. הם משלימים.

מסמכים נלווים: research/fabel5/V16_SPEC_HE.pdf (הפיזיקה והמתמטיקה המלאה) · V15_2_SPEC_HE.pdf · 1.8.2026